

0. 前言

OI-wiki 感觉说的并不通俗易懂啊.....导致它式子我还得自己手推一遍才能理解它的原理。

写一篇[面向普及组选手的Min_25筛](#)吧!

(不懂或有误可以指出并私信, 必关)

在学之前, 先膜拜 Min_25 大佬!!!

1. 问题

首先抛出一个问题: 给你一个**积性函数** f , 让你求 $\sum_{i=1}^n f(i)$, 也就是 $f(i)$ 的前缀和, 怎么做?

杜教筛通过构造一个 g 函数然后用卷积式子解决。

复杂度是 $O(n^{\frac{2}{3}})$ 。

而如果 $f(p)$ (p 为质数) 能快速计算并表达成若干个完全积性函数的和 (通常为关于 p 的多项式), 且 $f(p^c)$ 也可以快速计算时, 就可以使用 Min_25 筛。

完全积性函数的定义是,

$$\forall a, b, f(a)f(b) = f(ab)$$

注意, a, b 不需要互质!

2. 辅助函数

我们先定义, $G(k, n)$ 为:

从 2 到 n 遍历整数 i , 对于所有满足以下条件的 i , 对 $f(i)$ 求和。

条件: i 的最小质因子大于等于第 k 个质数。

用数学语言表述, 就是:

$$\sum_{i=2}^n f(i)[L(i) \geq P_k]$$

其中 $L(i)$ 就是 i 的最小质因子, P_k 就是第 k 个质数。

用代码表述, 就是:

```
int ans=0;
for(int i=2;i<=n;i++){
    if(L[i]>=pri[k]) ans+=f(i);
}
return ans;
```

边界就是当 $P_k > n$ 时 $G(k, n) = 0$ 。

3. 转化

求这个奇怪的东西有什么用？

先不管，看看怎么求。

首先，我们发现，只有 $L(i) \geq P_k$ 时， $f(i)$ 才对 $G(k, n)$ 产生贡献，所以我们枚举大于等于 P_k 的所有质数作为 i 的最小质因子。

具体怎么枚举呢？

对于每一个 P_j ，考虑 $L(i) = P_j$ ，那么就有两种情况：

1. P_j 在 i 中的次数等于 1，也就是说， i 因数分解后为 $\prod_{i=1}^k p_i^{c_i}$ ，则 $p_1 = P_j, c_1 = 1$ 。
。
2. P_j 在 i 中的次数大于 1，同理， $p_1 = P_j, c_1 > 1$ 。

第一种情况

对于所有满足第一种情况的 i ，它们的总贡献就是

$$f(P_j) + \sum_{t=2}^{\lfloor \frac{n}{P_j} \rfloor} f(tP_j)[L(t) \geq P_{j+1}]$$

为啥得到这个式子呢？

首先因为 i 是 j 的倍数，所以我们按照学莫反时推式子的原理，转而去枚举 $t = \frac{i}{P_j}$ 。

然后因为 j 是 i 的最小质因子，所以 t 的最小素因子一定大于 P_j ，或 $t = 1$ 。

所以这个式子的前半部分处理的是 $t = 1$ 的情况，就是 $f(P_j)$ 。（此时 $i = P_j$ ）

$t \neq 1$ 时，前面说过 t 的最小素因子一定大于 P_j ，即大于等于 P_{j+1} 。

所以 $L(t) \geq P_{j+1}$ 时， $f(i)$ ，也就是 $f(tP_j)$ ，才对答案构成贡献。

又因为 P_j 在 i 中的次数等于 1，所以 $\gcd(t, P_j) = 1$ ，又因为 f 积性，所以 $f(tP_j) = f(t)f(P_j)$ 。

所以原式变为：

$f(P_j) + f(P_j) \sum_{t=2}^{\lfloor \frac{n}{P_j} \rfloor} f(t)[L(t) \geq P_{j+1}]$ 根据 G 的定义，就可以写成：

$$f(P_j) + f(P_j)G(j+1, \lfloor \frac{n}{P_j} \rfloor)$$

注意！下面是关键点！

对于前面的一项 $f(P_j)$ ， $P_j \leq n$ 即可，但对于后面一项 $f(P_j)G(j+1, \lfloor \frac{n}{P_j} \rfloor)$ ，必须

$P_j^2 \leq n$ ，即 $t \geq P_j$ ，也就是说，当 $P_j^2 > n$ 时，后面那一项就是无用的。

因为后面那一项我们要求 $t \neq 1$ ，又 j 为 i 的最小素因子，所以必有 $t \geq P_j$ 。

第二种情况

对于所有满足第一种情况的 i ，它们的总贡献就是

$\sum_{c \geq 2, P_j^c \leq n} (f(P_j^c) + \sum_{t=2}^{\lfloor \frac{n}{P_j^c} \rfloor} f(tP_j^c)[L(t) \geq P_{j+1}])$ 我们枚举 c 作为 P_j 在 i 中的出现次数。

其余和第一种情况类似，也是枚举 $t = \frac{i}{P_j^c}$ 。

然后因为 j 是 i 的最小质因子，所以 t 的最小素因子一定大于 P_j ，或 $t = 1$ 。

所以这个式子的前半部分处理的是 $t = 1$ 的情况，就是 $f(P_j^c)$ 。(此时 $i = P_j^c$)

$t \neq 1$ 时，前面说过 t 的最小素因子一定大于 P_j ，即大于等于 P_{j+1} 。

所以 $L(t) \geq P_{j+1}$ 时， $f(i)$ ，也就是 $f(tP_j^c)$ ，才对答案构成贡献。

然后同理继续推式子：

$$\sum_{c \geq 2, P_j^c \leq n} \left(f(P_j^c) + f(P_j^c) \sum_{t=2}^{\lfloor \frac{n}{P_j^c} \rfloor} f(t)[L(t) \geq P_{j+1}] \right)$$

还是根据 G 的定义：

$$\sum_{c \geq 2, P_j^c \leq n} \left(f(P_j^c) + f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) \right)$$

贡献

那么，对于所有最小质因子为 P_j 的 i ， $f(i)$ 的贡献之和就是：

$$f(P_j) + f(P_j) G(j+1, \lfloor \frac{n}{P_j} \rfloor) + \sum_{c \geq 2, P_j^c \leq n} (f(P_j^c) + f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor))$$

此时，对于每一个大于等于 k 的 j ，计入贡献，则：

$$\begin{aligned} G(k, n) &= \left(\sum_{j \geq k, P_j \leq n} \left(f(P_j) + f(P_j) G(j+1, \lfloor \frac{n}{P_j} \rfloor) + \sum_{c \geq 2, P_j^c \leq n} (f(P_j^c) + f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)) \right) \right) \\ &= \left(\sum_{j \geq k, P_j^2 \leq n} \left(f(P_j) G(j+1, \lfloor \frac{n}{P_j} \rfloor) + \sum_{c \geq 2, P_j^c \leq n} (f(P_j^c) + f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)) \right) \right) + \sum_{j \geq k, P_j \leq n} f(P_j) \end{aligned}$$

这里说一下这一步是怎么推的。

由定义，当 $P_k > n$ 时， $G(k, n) = 0$ ，所以对于 $\lfloor \sqrt{n} \rfloor < P_j \leq n$ 的 P_j ，必有

$G(j+1, \lfloor \frac{n}{P_j} \rfloor) = 0$ ，所以不计入答案，又 $c \geq 2$ ，所以 $\lfloor \sqrt{n} \rfloor < P_j \leq n$ 时只有 $f(P_j)$ 那

一项会计入贡献，那干脆把它提出去单独处理好了。

开始化简式子，

$$= \left(\sum_{j \geq k, P_j^2 \leq n} \left(\sum_{c \geq 1, P_j^c \leq n} \left(f(P_j^c) + f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) \right) \right) - f(P_j) \right) + \sum_{j \geq k, P_j \leq n} f(P_j)$$

这一步就很初等，没什么好说的。

下一步还是很初等。

$$G(k, n) = \left(\sum_{j \geq k, P_j^2 \leq n} \sum_{c \geq 1, P_j^c \leq n} f(P_j^c)([c \neq 1] + G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)) \right) + \sum_{j \geq k, P_j \leq n} f(P_j)$$

设 $S(n) = \sum_{p \leq n, p \in \mathbb{P}} f(p)$ 。

则可以表示成：

$$G(k, n) = \left(\sum_{j \geq k, P_j^2 \leq n} \sum_{c \geq 1, P_j^c \leq n} f(P_j^c)([c \neq 1] + G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)) \right) + S(n) - S(P_{k-1})$$

终于我们推了这么久，得到了这个递推式子。

终止条件就是当 $P_k > n$ 是， $G(k, n) = 0$ 。

而刚才那么多的分析，在 OI-wiki 中就是一个直接的等式，一笔带过。

所以我说 OI-wiki 不好懂。

这里有一个艾佛森括号，很丑。

怎么把它干掉？

简单！

我们知道，对于枚举到的最后一个 c ，此时 $P_j^c \leq n$ ， $P_j^{c+1} > n$ ，则有 $\lfloor \frac{n}{P_j^c} \rfloor < P_j < P_{j+1}$ 。

所以根据定义，此时的 $G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) = 0$ 。

所以我们就知道 c 枚举到最后一个的时候贡献只有 $f(P_j^c)$ 。

于是我们可以发现，根据乘法分配律， $f(P_j^c)[c \neq 1]$ 这一项在 $c = 1$ 时没有贡献。

而 $f(P_j^c)G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)$ 这一项在最后一个 c 时没有贡献。

所以我们可以枚举时将 $\sum_{c \geq 1, P_j^c \leq n} f(P_j^c)([c \neq 1] + G(j+1, \lfloor \frac{n}{P_j^c} \rfloor))$ ，

改为 $\sum_{c \geq 1, P_j^{c+1} \leq n} \left(f(P_j^c)G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) + f(P_{j+1}^c) \right)$ 。

这一步很重要，务必理解透彻！

如果您还不理解，举个例子：

$$\sum_{i=1}^n (a_i + b_i) - a_1 - b_n = \sum_{i=1}^{n-1} (a_{i+1} + b_i)$$

如果您理解了，继续推：

于是有

$$G(k, n) = \left(\sum_{j \geq k, P_j^2 \leq n} \sum_{c \geq 1, P_j^{c+1} \leq n} \left(f(P_j^c)G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) + f(P_{j+1}^c) \right) \right) + S(n) - S(P_{k-1})$$

假设 S 我们已经会计算了，那么这不就是一个 DP 式子吗？

直接模拟原式即可。

4. 求出 S

现在考虑怎么求 S 。

我们知道 Min_25 筛处理的 f 一般满足这样的性质：

对于质数 p ，有 $f(p)$ 是 p 的一个低次多项式。

设该多项式为：

$$\sum_{i=1}^r a_i p^{c_i}$$

r 就是项数， c_i 严格递增。（废话）

$$\text{由 } S(n) = \sum_{p \leq n, p \in \mathbb{P}} f(p),$$

得到：

$$S(n) = \sum_{p \leq n, p \in \mathbb{P}} \sum_{i=1}^r a_i p^{c_i} = \sum_{i=1}^r a_i \sum_{p \leq n, p \in \mathbb{P}} p^{c_i}$$

现在考虑对于每个 p^{c_i} 求贡献。

设 $E(k, n)$ 为： $\sum_{i=2}^n [P_k < L(i) \text{ or } i \in \mathbb{P}] i^s$ ，也就是埃氏筛在筛掉 k 轮之后剩下的所有 i

的 s 次方的和。

你不会埃氏筛？

好吧，既然是一篇面向普及组选手的博客，我还是说一下吧。

埃氏筛就是说，先筛掉 2 的倍数，再筛掉 3 的倍数，再筛掉 5 的倍数，以此类推，直到筛完小于等于 $\lfloor \sqrt{n} \rfloor$ 的最大质数的倍数时， n 及以下没被筛掉的数就都是质数。

注意这里的“倍数”都是大于 2 的倍数。

为什么是 $\lfloor \sqrt{n} \rfloor$ 呢？

考虑如果有一个 n 以内有一个合数 A 大于 $\lfloor \sqrt{n} \rfloor$ ，这就代表着它必有一个因数小于 $\lfloor \sqrt{n} \rfloor$ ，所以它一定已经被筛掉了。

你忘了什么是 $P_k, L(i)$? —(你是真行)—

再提一遍， $L(i)$ 就是 i 的最小质因子， P_k 就是第 k 个质数。

既然如此，我们就可以得出，对于每个 $s = c_i$ ，它对 $S(n)$ 的贡献就是 $E(l, n)$ 。

其中 P_l 为小于等于 $\lfloor \sqrt{n} \rfloor$ 的最大质数。

现在就需要求出 $E(k, n)$ 。

首先，根据定义， $E(k-1, n)$ 就是埃氏筛在筛掉 $k-1$ 轮之后剩下的所有 i 的 s 次方的和。

我们就需要考虑第 k 轮筛完之后让 $E(k-1, n)$ 的值减去了多少。

我们知道第 k 轮一定筛去了 P_k 的倍数，而且。

这个减去的量就是：（开始推式子）

$$\begin{aligned} &= \sum_{i=2}^{\lfloor \frac{n}{P_k} \rfloor} [P_{k-1} < L(i) \text{ or } i \in \mathbb{P}] (i P_k)^s \\ &= P_k^s \sum_{i=2}^{\lfloor \frac{n}{P_k} \rfloor} [P_{k-1} < L(i) \text{ or } i \in \mathbb{P}] i^s \\ &= P_k^s E(k-1, \lfloor \frac{n}{P_k} \rfloor) \end{aligned}$$

你一定会觉得这个式子有问题，或是看不懂，对吧？

这个式子真对吗？

首先，这个式子的意思是：对于 n 以内所有的 P_k 的倍数 m ，若 $L(\frac{m}{P_k})$ 大于等于 P_k （这就代表 $L(m)$ 也大于等于 P_k ）或者是 m 是 P_k 的质数倍，那么答案加上 $(mP_k)^s$ ，这就是 $E(k, n)$ 相对于 $E(k-1, n)$ 所减去的量。

乍一看，好像没问题啊，就是埃氏筛第 k 轮所筛出的数的 s 次方的和啊，仔细一想，不对劲，如果 m 是 P_k 的质数倍，且这个质数小于 P_k ，那么 m 应该在之前就被筛掉了，怎么会留到现在呢？

所以说这个式子减多了。

怎么把这个式子加回来呢？

我们发现，减多了的数都可以被表示成如下式子： $P_v P_k$ ，其中 $v < k$ 。

那我们就枚举 P_v 及以内的数：

$$\sum_{i=2}^{P_{k-1}} [P_{k-1} < L(i) \text{ or } i \in \mathbb{P}] (iP_k)^s$$

我们发现 P_{k-1} 不可能小于 $L(i)$ ，所以原式就是：

$$\sum_{i=2}^{P_{k-1}} [i \in \mathbb{P}] (iP_k)^s$$

这就是被减多了的式子。

你发现它就是： $P_k^s E(k-1, P_{k-1})$ 。

所以就可以得到最终的递推式：

$$E(k, n) = E(k-1, n) - P_k^s (E(k-1, \lfloor \frac{n}{P_k} \rfloor) - E(k-1, P_{k-1}))$$

但是这个式子还是不一定对。

当 $P_k > \sqrt{n}$ 时，第 k 遍埃氏筛是不会筛掉任何数的。

所以真正的式子是：

$$E(k, n) = E(k-1, n) - P_k^s [P_k^2 \leq n] (E(k-1, \lfloor \frac{n}{P_k} \rfloor) - E(k-1, P_{k-1}))$$

这个式子就是对的了。

最后递推的终止条件是 $E(0, x) = \sum_{i=2}^x i^s$ 。（我们假定第 0 个质数是 1）

那我们可以求出 E 了，也就可以求出 S 了。

也就可以求出 G 了。

我们发现一个性质： $\sum_{i=1}^n f(i) = G(1, n) + f(1) = G(1, n) + 1$ 。

于是我们就基本学会了 Min_25 筛的大致步骤。

5. 复杂度分析

在这里我不说具体复杂度是如何计算的，我在此想补充一些具体计算时的性质。

优化

重新回顾前面 $G(k, n)$ 的递推式：

$$G(k, n) = \left(\sum_{j \geq k, P_j^2 \leq n} \sum_{c \geq 1, P_j^{c+1} \leq n} \left(f(P_j^c) G(j+1, \lfloor \frac{n}{P_j^c} \rfloor) + f(P_j^{c+1}) \right) \right) + S(n) - S(P_{k-1})$$

因为只有 $P_j^2 \leq n$ 时两个 Σ 才计入贡献，才会计算 $G(j+1, \lfloor \frac{n}{P_j^c} \rfloor)$ ，因此可以得出在所有需要算出的 $G(A, B)$ 中， P_A 必然小于等于 $\lfloor \sqrt{n} \rfloor$ ， P_{A-1} 则必然小于 $\lfloor \sqrt{n} \rfloor$ 。

然后我们有一个引理：

$$\lfloor \frac{\lfloor \frac{n}{x} \rfloor}{y} \rfloor = \lfloor \frac{n}{xy} \rfloor$$

所以 B 的所有取值都是某个 $\lfloor \frac{n}{\lambda} \rfloor$ 。

我们又有有一个引理：

$\forall \lambda \leq n, \lfloor \frac{n}{\lambda} \rfloor$ 最多只有 $2\sqrt{n}$ 种取值： $1, 2, \dots, \lfloor \sqrt{n} \rfloor, \lfloor \frac{n}{\sqrt{n}-1} \rfloor, \dots, \lfloor \frac{n}{2} \rfloor, n$ 。

又因为每次计算递推式时 S 函数只需要在 P_{A-1}, B 点的取值就够了，所以在整个过程中， S 所需的取值点就只有 $1, 2, \dots, \lfloor \sqrt{n} \rfloor, \lfloor \frac{n}{\sqrt{n}-1} \rfloor, \dots, \lfloor \frac{n}{2} \rfloor, n$ 这 $O(\sqrt{n})$ 个点。

但是 n 太大，你可以选择 `unordered_map`，但是常数更小的方法是离散化然后存。

关于时间复杂度中求和与积分的转换

如果您不会积分可以跳过这一步，对算法学习无影响。

我们发现，在许多算法的复杂度推导中，会发现这么一步：

（注意假定 f 是连续，恒为正，且可积的）

$$O(\sum_{i=1}^n f(i)) = O(\int_0^n f(x) dx)$$

为什么这么做是正确的？

有的人会说，因为求和就是离散的积分啊。

确实这么说也是对的，但是证一下更严谨。

即证：

$$\Theta(\sum_{i=1}^n f(i)) = \int_0^n f(x) dx$$

设 $\sigma(n) = \sum_{i=1}^n f(i), \rho(n) = \int_0^n f(x)dx$ 。

即证：

$$\exists N, c_1, c_2 > 0, \forall n \geq N, n \in \mathbb{N}^+, c_1 \sigma(n) \leq \rho(n) \leq c_2 \sigma(n)$$

不妨直接证明更强的性质 $N = 0$ 时即满足。

发现：

$$\begin{aligned} c_1 \sigma(n) &\leq \rho(n) \leq c_2 \sigma(n) \\ \iff c_1 \sum_{i=1}^n f(i) &\leq \int_0^n f(x)dx \leq c_2 \sum_{i=1}^n f(i) \end{aligned}$$

那么我们只需要证，对 $\forall i \in \mathbb{N}^+, 1 \leq i \leq n$ ，有 $u_i f(i) \leq \int_{i-1}^i f(x)dx \leq v_i f(i)$ 。

然后 c_1 取 $\min_{i=1}^n u_i$ ， c_2 取 $\max_{i=1}^n v_i$ 即可。

由积分中值定理以及 f 连续可知，存在 $\xi \in [i-1, i]$ ，使得 $\int_{i-1}^i f(x)dx = f(\xi)$ ，

取 $u_i = \frac{f(\xi)}{f(i)} - \varepsilon, v_i = \frac{f(\xi)}{f(i)} + \varepsilon$ 即可。

得证。

由此我们有 min_25 筛的复杂度是 $O(\frac{n^{\frac{3}{4}}}{\log n})$ 或 $O(n^{1-\varepsilon})$ ，想看具体推导就见 OI-wiki 或者大佬们的博客吧。

6. 具体实现

这里就以模板为例。

```
#include <bits/stdc++.h>
#define sor(i, l, r) for (int i = l; i <= r; i++)
#define int __int128
using namespace std;
/* 快读部分，暂时不予展示 */
namespace Revitalize
{
    const int N = 2e6+7, P = 1e9 + 7;
    int n,m;
    int Ex01[N], Ex02[N];
    int ids[N], sqrs[N];

    bool vis[N], cuur;
    int pri[N],cl[N],I[N],II[N];
    inline void LinearSieve(int lim)
    {
```

```

ids[1] = sqrs[1] = 1;
for (int i = 2; i <= lim; i++)
{
    if (!vis[i])
    {
        pri[++pri[0]] = i;
        ids[pri[0]] = (ids[pri[0] - 1] + i) % P;
        sqrs[pri[0]] = (sqrs[pri[0] - 1] + i * i) % P;
    }
    for (int j = 1; j <= pri[0] && i * pri[j] <= lim; j++)
    {
        vis[i * pri[j]] = 1;
        if (i % pri[j] == 0)
        {
            break;
        }
    }
}
}

inline int L(int x){
    return (x <= sqrt((double)n)) ? (I[x]) : (II[n/x]);
}

inline void Ex(int n)
{
    for (int k = 1; pri[k] * pri[k] <= n; k++)
    {
        for (int i = 1; i <= m && pri[k] * pri[k] <= cl[i]; i++)
        {
            Ex01[i] = (Ex01[i] - pri[k] * (Ex01[L(cl[i] /
pri[k])] - ids[k - 1]) % P + P + P) % P;
            Ex02[i] = (Ex02[i] - pri[k] * pri[k] % P *
(Ex02[L(cl[i] / pri[k])] - sqrs[k - 1]) % P + P + P) % P;
        }
    }
}

int G(int k, int n)
{
    if (pri[k] > n)
    {
        return 0;
    }
    int ans = 0;
    for (int j = k; pri[j] * pri[j] <= n; j++)
    {

```

```

        for (int Pjc = pri[j]; Pjc * pri[j] <= n; Pjc *=
pri[j])
        {
            ans += Pjc % P * (Pjc - 1 + P) % P * G(j + 1, n /
Pjc) % P;

            ans %= P;
            ans += (Pjc % P * pri[j] % P) * (Pjc * pri[j] % P -
1 + P) % P;

            ans %= P;
        }
    }
    ans += (Ex02[L(n)] - Ex01[L(n)] + P) % P;
    ans -= (sqrs[k - 1] - ids[k - 1] + P) % P;
    return ans;
}

inline void Lx(int n)
{
    int l = 1, r = 0;
    while (l <= n) {
        r = n / (n / l);
        cl[++m] = n / l;
        Ex01[m] = cl[m] * (cl[m] + 1) / 2 % P - 1;
        Ex02[m] = cl[m] * (cl[m] + 1) % P * (cl[m] + cl[m] + 1)
% P * 166666668 % P - 1;
        if (cl[m] <= sqrt((double)(n)))
            I[cl[m]] = m;
        else
            II[n / cl[m]] = m;
        l = r + 1;
    }
}

inline void work()
{
    read(n);
    LinearSieve((int)(1000000));
    Lx(n);
    Ex(n);
    int ans = G(1, n) % P + 1;
    print(ans);
}

}

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);

```

```
return Revitalize::work(), 0;  
}
```

顺便警示一下后人，这题需要 `__int128`，而且要随时随地取模。

7. 优化

这里我知道有三种优化：

- [Min_26](#) 筛，使用根号分治优化。
- [新版 Min_25](#) 筛，然而公式炸了，似乎是树状数组优化？
- [zhoukangyang](#) 的算法，优化到了 $O(\sqrt{n}\text{polylog}(n))$ ，太强了！
这像是我能会的吗？—

Inf. 鸣谢

感谢 [luuia](#) 提供的高质量模拟赛使得我学习这一算法。

感谢 [Collapsarr](#) 提出在赛时当场学当场写的新思路。（尽管没有实现）

感谢神的光环照耀。

感谢 CWH 提供精神支持。